INSTALLATION GUIDE

Wonderful Minds Installation & Setup Guide

A complete manual for installing, running, and deploying your calm learning game for children with autism

Version	Stack	Games
1.0	**Next.js 16**	**16 Activities**

MADE WITH LOVE FOR EVERY WONDERFUL MIND

CONTENTS

Table of Contents

CHAPTER 01

Introduction & What's Included

Welcome to **Wonderful Minds** — a calm, sensory-friendly learning playground designed especially for children with autism, ADHD, and other neurodivergent needs. This guide will walk you through everything you need to install, run, and deploy the game so you can start sharing it with families right away.

Wonderful Minds is built as a modern web application, which means it runs in any web browser — on phones, tablets, laptops, and desktop computers. There is nothing for parents to install; they simply open a link and the game starts. Your job is to set up the project, customize it if you like, and deploy it to the internet so families can access it.

What's in the Package

When you downloaded the project, you received a complete, production-ready application. Here is what each part does:

📁 Source Code

All 16 game components, the home screen, virtual pet system, achievements, and daily challenges — written in TypeScript with full type safety.

🎨 Assets & Sounds

All game sounds are synthesized in real-time using the Web Audio API (no audio files needed). Emoji-based visuals keep the package lightweight.

💾 Progress System

Children's stars, badges, pet progress, and streaks are saved automatically in the browser's localStorage — no database setup required.

📖 This Manual

Step-by-step instructions for installation, development, building, and deployment.

Technology Stack

The project uses a modern, well-supported technology stack that will receive security updates for years to come:

Technology	Version	Purpose
Next.js	16.x	Web framework (React-based)
TypeScript	5.x	Type-safe JavaScript
Tailwind CSS	4.x	Styling system
Framer Motion	12.x	Smooth animations
Zustand	5.x	State management & progress saving
Web Audio API	Built-in	Sound generation (no files needed)
Speech Synthesis	Built-in	Spoken instructions

Who is this manual for? This guide is written for anyone who downloaded the source code — whether you are a developer, a seller preparing to deploy the game for customers, or a parent comfortable with following technical instructions. No prior React experience is needed; just follow each step in order.

CHAPTER 02

System Requirements & Prerequisites

Before you begin, make sure your computer meets the minimum requirements. The game itself is lightweight, but the development tools need some resources to run smoothly. The following specs apply to any operating system — Windows, macOS, or Linux.

Hardware Requirements

Component	Minimum	Recommended
RAM	4 GB	8 GB or more
Storage	500 MB free	1 GB free
Processor	Any dual-core CPU	Modern multi-core
Internet	Required for setup	Broadband for deployment

Operating System

The project works on any modern operating system. The three most common are fully supported:

- **Windows** 10 or 11 (64-bit) — Windows 8.1 works but is not recommended
- **macOS** 11 (Big Sur) or newer — Intel and Apple Silicon (M1/M2/M3) both work
- **Linux** — Ubuntu 20.04+, Debian 11+, Fedora 35+, or any distribution with Node.js support

Required Software

You will need to install three pieces of software before you can run the project. Chapter 3 walks through installing each one, but here is an overview so you know what to expect:

① Node.js (v18 or newer)

The JavaScript runtime that powers the development server and production build. Required for any Next.js project. Download from nodejs.org.

② Bun (recommended) or npm

A fast package manager. Bun is 10-30× faster than npm and is what this guide uses. npm comes bundled with Node.js as a fallback. Get Bun from bun.sh.

③ Git

Version control software. Needed if you plan to deploy to Vercel or Netlify (they connect via Git). Download from git-scm.com.

Optional Software

- **VS Code** — Free code editor from Microsoft. Excellent TypeScript and Next.js support. Download from code.visualstudio.com.
- **Vercel CLI** — For command-line deployment to Vercel. Install with `npm i -g vercel`.
- **Docker** — Only if you plan to self-host using containers. Install from docker.com.

> **Tip:** If this is your first time setting up a development environment, allow about 30-45 minutes for the whole process. Once everything is installed, future projects will be ready in minutes.

CHAPTER 03

Install Prerequisites

This chapter walks you through installing Node.js, Bun, and Git on your computer. Follow the section for your operating system. If you already have these tools installed, skip to Chapter 4 — but verify your versions match the requirements using the commands shown here.

Step 1: Install Node.js

Node.js is the JavaScript runtime that runs the development server and builds the production version of your game. You need version 18 or newer.

🪟 Windows

1. Go to **nodejs.org**
2. Download the "LTS" (Long Term Support) version
3. Run the installer (.msi file)
4. Accept all default options — click Next through each screen
5. Verify installation by opening **Command Prompt** or **PowerShell** and typing:

```
# Verify Node.js installation
node --version
# Expected output: v18.x.x or higher (e.g., v20.11.0)

npm --version
# Expected output: 9.x.x or higher
```

🍎 macOS

Option A — Installer (easiest): Download the LTS version from nodejs.org and run the .pkg installer.

Option B — Homebrew (recommended for developers): Open Terminal and run:

```
# Install Node.js via Homebrew
brew install node

# Verify installation
node --version
# Expected: v18.x.x or higher
```

🐧 Linux (Ubuntu/Debian)

Open a terminal and run the following commands. Using NodeSource ensures you get a recent version rather than the older one in default repositories.

```
# Add NodeSource repository for Node.js 20.x
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -

# Install Node.js (includes npm)
sudo apt-get install -y nodejs

# Verify installation
node --version
npm --version
```

> **Important:** If `node --version` shows anything below v18, you need to upgrade. Older versions of Node.js will not run Next.js 16 correctly and you will see errors during installation or startup.

Step 2: Install Bun (Recommended Package Manager)

Bun is a modern, extremely fast package manager and runtime. It installs dependencies 10-30× faster than npm and is what this guide uses throughout. Installing it is optional but highly

recommended — if you skip it, replace every `bun` command in this guide with `npm` (or `npx` for commands that run executables).

```
# Install Bun — works on Windows, macOS, and Linux# macOS / Linux:
curl -fsSL https://bun.sh/install | bash

# Windows (PowerShell):
powershell -c "irm bun.sh/install.ps1 | iex"

# Verify installation
bun --version
# Expected output: 1.x.x
```

> **No Bun?** If you prefer npm, that is completely fine. Every `bun install` becomes `npm install`, every `bun run dev` becomes `npm run dev`, and so on. The project works identically with either.

Step 3: Install Git

Git is version control software. You need it if you plan to deploy to Vercel or Netlify (both services connect to your code via Git). If you only plan to run the game locally, you can skip this step.

🪟 Windows

Download Git from **git-scm.com** and run the installer. Accept all default options. This also installs "Git Bash", a useful terminal.

🍎 macOS

Option A: Install Xcode Command Line Tools: `xcode-select --install` in Terminal (includes Git).
Option B: Use Homebrew: `brew install git`

🐧 Linux

Git is usually pre-installed. If not: `sudo apt-get install git` (Ubuntu/Debian) or `sudo dnf install git` (Fedora).

```
# Verify Git installation
git --version
# Expected output: git version 2.30.0 or higher
```

All set? If `node`, `bun` (or `npm`), and `git` all return version numbers without errors, you are ready to move on to Chapter 4 and download the project.

CHAPTER 04

Download & Extract the Project

After purchasing or downloading Wonderful Minds, you received a ZIP file containing the complete source code. This chapter shows you how to extract it and understand the folder structure.

Step 1: Extract the ZIP File

Locate the ZIP file you downloaded (usually named `wonderful-minds.zip`) and extract it to a folder where you want to keep the project. A good location is your Documents folder or a dedicated "Projects" folder.

🪟 Windows

Right-click the ZIP file → **Extract All...** → Choose a destination → Click **Extract**.

🍎 macOS

Double-click the ZIP file. macOS will automatically extract it into a folder with the same name in the same location.

🐧 Linux

Right-click → **Extract Here**, or in terminal: `unzip wonderful-minds.zip`

Step 2: Open a Terminal in the Project Folder

All the commands in the rest of this guide need to be run from inside the project folder. Open a terminal and navigate to where you extracted the files.

```
# Navigate to the project folder# Replace the path below with wherever you
extracted the ZIP# Example on macOS/Linux:
cd ~/Documents/wonderful-minds

# Example on Windows:
cd C:\Users\YourName\Documents\wonderful-minds

# Verify you are in the right place
ls
# You should see: package.json, src/, public/, prisma/, etc.
```

Step 3: Understand the Folder Structure

Here is what each folder and file does. You do not need to memorize this — just refer back if you need to find something later.

```
wonderful-minds/
├── src/# Main source code
│   ├── app/# Next.js app router (pages)
│   │   ├── page.tsx          # Home page - main game router
│   │   ├── layout.tsx        # Root layout with metadata
│   │   └── globals.css       # Global styles
│   ├── components/# React components
│   │   ├── games/# All 16 game components
│   │   ├── home-screen.tsx   # Home screen with activity grid
│   │   ├── virtual-pet.tsx   # Pet companion system
│   │   └── ...
│   ├── lib/# Utilities and data
│   │   ├── game-store.ts     # Zustand store (progress, pet, badges)
│   │   ├── games-data.ts     # All game content (animals, letters, etc.)
│   │   └── sound.ts          # Web Audio sound generator
│   └── hooks/# Custom React hooks
├── public/# Static assets (logo, robots.txt)
├── prisma/# Database schema (optional)
├── package.json# Project dependencies and scripts
├── tsconfig.json# TypeScript configuration
├── next.config.ts# Next.js configuration
└── tailwind.config.ts# Tailwind CSS configuration
```

> **Quick tip:** The only folder you will ever need to edit is `src/`. Everything in there is the game; everything outside is configuration. Chapter 10 explains how to customize the game content.

CHAPTER 05

Install Dependencies

The project relies on dozens of open-source libraries (React, Framer Motion, Tailwind, Zustand, and more). These are listed in `package.json` but are not included in the ZIP file — you need to download them. This is a one-time step that takes 1-3 minutes depending on your internet speed.

Step 1: Run the Install Command

Make sure your terminal is in the project folder (see Chapter 4), then run:

```
# Using Bun (recommended — faster)
bun install

# OR using npm (if you did not install Bun)
npm install
```

You will see a lot of text scroll by as packages are downloaded and installed. This is normal. When it finishes, you should see a success message and a new `node_modules/` folder will appear in your project directory.

What to Expect

✅ Success Looks Like

With Bun: A summary showing how many packages were installed and the total time (usually 5-15 seconds).
With npm: A line like `added 312 packages in 47s` with no errors.

Common Issues & Fixes

❌ "command not found: bun" or "npm"

Node.js or Bun was not installed correctly. Go back to Chapter 3, install them, then close and reopen your terminal (terminals need to be restarted to recognize new commands).

❌ "EACCES: permission denied"

This happens on macOS/Linux when npm tries to write to a system folder. Fix: run `sudo chown -R $USER ~/.npm` then retry. Or better, use Bun which installs to your home directory.

❌ **Network errors / timeouts**

Check your internet connection. If you are behind a corporate proxy, configure npm: `npm config set proxy http://your-proxy:port`. Try again — sometimes a single package fails temporarily.

Did it work? If you see a `node_modules/` folder in your project directory (it will be large, 200-400 MB), the installation succeeded. You are ready to run the game!

CHAPTER 06

Run the Development Server

Now for the exciting part — actually running the game! The development server lets you play the game in your browser and automatically reloads whenever you make changes to the code. This is what you will use while customizing or testing.

Step 1: Start the Server

In your terminal (still in the project folder), run:

```
# Using Bun
bun run dev

# OR using npm
npm run dev
```

After a few seconds, you will see output like this:

```
# Expected terminal output:
  ▲ Next.js 16.1.3 (Turbopack)
  - Local:        http://localhost:3000
  - Network:      http://192.168.1.xxx:3000

  ✓ Ready in 980ms
```

Step 2: Open the Game in Your Browser

Open your web browser (Chrome, Firefox, Safari, or Edge all work) and go to:

```
http://localhost:3000
```

You should see the Wonderful Minds home screen with the rainbow logo, a welcome message, your virtual pet, and a grid of 16 colorful game cards. Click any card to play that game!

Step 3: Testing on Other Devices (Optional)

If you want to test the game on your phone or tablet while developing, make sure both devices are on the same Wi-Fi network. Then use the "Network" address shown in the terminal instead of localhost. For example, if your computer's IP is 192.168.1.50, open `http://192.168.1.50:3000` on your phone.

Step 4: Stop the Server

When you are done playing, go back to your terminal and press **Ctrl + C** (Windows/Linux) or **Control + C** (macOS). This stops the server safely.

> **Hot Reload:** While the dev server is running, any change you make to a file (colors, text, game logic) instantly appears in the browser. You do not need to restart the server or refresh the page. This makes customizing the game very fast.

CHAPTER 07

Build for Production

The development server is great for testing, but it is not optimized for real users. Before you deploy the game to the internet, you need to create a production build. This process optimizes the code, minifies files, and removes development-only features. The result is faster page loads and a smaller download for families using the game.

Step 1: Create the Build

Stop the development server first (Ctrl + C), then run:

```
# Create an optimized production build
bun run build

# OR with npm
npm run build
```

This takes 30-90 seconds. You will see output like this:

```
# Expected output (abbreviated):
  ▲ Next.js 16.1.3 (Turbopack)
  Creating an optimized production build ...

  ✓ Compiled successfully
  ✓ Collecting page data
  ✓ Generating static pages (8/8)
  ✓ Finalizing page optimization

  Route (app)                              Size     First Load JS
  ┌ ○ /                                    12.4 kB         98.7 kB
  └ ○ /_not-found                          871 B           87.1 kB
  + First Load JS shared by all            86.3 kB

  ○  (Static)  prerendered as static content
```

Step 2: Test the Production Build Locally

Before deploying, it is a good idea to test the production build on your own computer to make sure everything works:

```
# Start the production server
bun run start

# OR with npm
npm run start
```

Then open `http://localhost:3000` in your browser. The game should look and play exactly the same, but pages will load faster. Test a few games to make sure everything works. Press Ctrl + C to stop when done.

> **What changed?** The production build creates a `.next/` folder with optimized files. This is what gets deployed to the internet. The `.next/` folder should never be edited manually — always rebuild after making code changes.

CHAPTER 08

Deployment Options

Deployment means putting your game on the internet so families can access it from anywhere. There are three main options, ranging from easiest to most advanced. Pick the one that fits your needs and technical comfort level.

Option	Difficulty	Cost	Best For
A. Vercel	★☆☆ Easiest	Free tier	Most users — recommended
B. Netlify	★★☆ Easy	Free tier	Alternative to Vercel
C. Self-host	★★★ Advanced	$5-20/mo VPS	Full control, custom domain

Option A: Deploy to Vercel (Recommended)

Vercel is the company that created Next.js, so they offer the best integration. The free tier handles thousands of visitors per month — more than enough for most uses. Deployment takes about 5 minutes.

1. **Create a GitHub account** (if you don't have one) at github.com. Then create a new repository and upload your project files to it. You can use GitHub's web interface or the GitHub Desktop app.
2. **Go to vercel.com** and sign up using your GitHub account. Click **"Add New..."** → **"Project"**.
3. **Import your repository.** Vercel will list your GitHub repos. Click "Import" next to your wonderful-minds repo.
4. **Configure (defaults are fine).** Vercel auto-detects Next.js. Leave all settings as default and click **"Deploy"**.
5. **Wait 2-3 minutes.** Vercel builds and deploys your game. When done, you get a URL like `wonderful-minds.vercel.app`. Share this with families!

> **Custom domain:** On Vercel, go to Project Settings → Domains to add a custom domain like `wonderfulminds.com`. You will need to buy the domain (about $10-15/year from Namecheap or Google Domains) and update its DNS to point to Vercel.

Option B: Deploy to Netlify

Netlify is another popular hosting platform with a generous free tier. The process is very similar to Vercel:

1. Push your code to GitHub (same as Vercel step 1).
2. Go to **netlify.com** and sign up with GitHub.
3. Click **"Add new site"** → **"Import an existing project"**.
4. Select your repo. Set build command to `npm run build` and publish directory to `.next`. Click **"Deploy site"**.
5. Wait 2-3 minutes. Netlify gives you a URL like `wonderful-minds.netlify.app`.

Option C: Self-Hosting (Advanced)

If you want full control — perhaps you already have a server, or you want to avoid third-party platforms — you can host the game yourself. You will need a VPS (Virtual Private Server) from DigitalOcean, Linode, Hetzner, or similar (starting around $5/month).

Using PM2 (process manager):

```
# On your server, install Node.js and PM2
npm install -g pm2

# Clone your repo and install deps
git clone https://github.com/yourname/wonderful-minds.git
cd wonderful-minds
npm install
npm run build

# Start the production server with PM2
pm2 start npm --name "wonderful-minds" -- start

# Make it restart on server reboot
pm2 startup
pm2 save
```

Using Docker (alternative):

```
# Create a Dockerfile in your project root:
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build
EXPOSE 3000
CMD ["npm", "start"]

# Build and run the Docker image
docker build -t wonderful-minds .
docker run -p 80:3000 -d wonderful-minds
```

For self-hosting, you will also want to set up Nginx as a reverse proxy and install SSL certificates (free via Let's Encrypt) for HTTPS. Search for "Nginx reverse proxy Next.js" for detailed tutorials.

CHAPTER 09

Troubleshooting Common Issues

If something goes wrong, do not panic! Most issues have simple fixes. This chapter covers the most common problems. Always start by reading the error message in your terminal — it usually tells you exactly what is wrong.

Installation & Startup Issues

❌ "Port 3000 is already in use"

Cause: Another program (often another Node.js project) is using port 3000.
Fix: Either close the other program, or run the game on a different port: `bun run dev -- -p 3001` then open `http://localhost:3001`

❌ "Module not found" or "Cannot find package"

Cause: Dependencies were not installed, or the install was interrupted.
Fix: Delete the `node_modules` folder and `bun.lock` / `package-lock.json`, then run `bun install` again from scratch.

❌ "Node.js version is not supported"

Cause: Your Node.js is older than version 18.
Fix: Uninstall old Node.js, download and install the latest LTS version from nodejs.org. Restart your terminal, then verify with `node --version`.

❌ White/blank screen in browser

Cause: Usually a JavaScript error. Open your browser's Developer Tools (F12 or Cmd+Option+I) and check the Console tab for red error messages.
Fix: The most common cause is a corrupted localStorage from an old version. In the browser console, run: `localStorage.clear()` then refresh the page.

Game Behavior Issues

🔇 No sound in the games

Cause: Browsers block audio until the user interacts with the page (a security feature). Also check the sound toggle button in the top-right of the home screen.
Fix: Click anywhere on the page first, then play a game. Make sure the speaker icon (top-right) does not have an X. Check your device's volume and that the browser tab is not muted.

🔄 Progress (stars, pet) keeps resetting

Cause: Progress is stored in the browser's localStorage. If you use private/incognito mode, or clear your browser data, progress is lost. Different browsers and devices do not sync.
Fix: Use a normal (non-private) browser window. Do not clear site data. This is by design — for a children's game, local storage is simpler and more privacy-friendly than user accounts.

📱 Game doesn't fit on phone screen

Cause: The game is responsive but some very small screens may need to scroll.
Fix: Make sure you are not zoomed in. Double-tap to reset zoom. The game is designed for screens 360px wide and up. Rotate to landscape for tablets.

Build & Deployment Issues

❌ Build fails with "Type error"

Cause: A TypeScript type mismatch in the code. This should not happen with the original package, but may occur if you edited files.
Fix: Read the error — it tells you the file and line number. Undo your recent changes, or fix the type error shown. Run `bun run lint` to find issues.

🌐 Deployment shows 404 error

Cause: The platform cannot find the built files, or the build command failed during deployment.
Fix: Check the deployment logs on Vercel/Netlify. Make sure the build command is `npm run build` (or `bun run build`) and the output directory is `.next`.

Still stuck? If none of these solutions work, try the universal fix: delete the `node_modules` folder and `.next` folder, run `bun install` again, then `bun run dev`. This solves 80% of mysterious issues.

CHAPTER 10

Customization & Support

Wonderful Minds is designed to be customizable. You can change colors, add new games, modify content, or rebrand it for your needs. This chapter introduces the most common customizations and points you to the right files.

Change the Game's Colors

The color palette is defined in `src/app/globals.css`. Look for the `:root` section near the top. Each color is an oklch value — you can replace these with hex colors from any color picker. The home screen and game cards use Tailwind gradient classes like `from-amber-200` which you can change in `src/lib/game-store.ts` (the `activityMeta` object).

Add or Edit Game Content

All game content (animals, emotions, letters, shapes, etc.) lives in one file: `src/lib/games-data.ts`. This file is well-organized with clear sections. To add a new animal, for example, find the `animals` array and add a new object following the same format. The game will automatically include it.

> **Example:** To add a "Turtle" to the Animal Friends game, open `src/lib/games-data.ts`, find the `animals` array, and add: `{ id: 'turtle', name: 'Turtle', emoji: '🐢', sound: 'Slow and steady!', habitat: 'Pond' }` — save the file and the game updates instantly.

Change the Pet's Default Name

Open `src/lib/game-store.ts` and find the `defaultPet` object. Change `name: 'Star'` to whatever you like. You can also change the default pet type from 'cat' to 'dog', 'bunny', or 'dragon'.

Reset a User's Progress

All progress is stored in the browser. To reset everything, click the refresh icon (top-right of home screen) and confirm. Or, in the browser console (F12), run `localStorage.clear()` and refresh.

Add a New Game

Adding a completely new game requires some React/TypeScript knowledge, but follows a clear pattern:

1. Create a new file in `src/components/games/` — copy an existing simple game like `body-parts-game.tsx` as a starting template.

2. Add the game's ID to the `ActivityId` type and `activityMeta` in `src/lib/game-store.ts`.
3. Add the game's content to `src/lib/games-data.ts`.
4. Import and route the game in `src/app/page.tsx`.

Need More Help?

If you run into issues not covered in this manual, here are some resources:

- **Next.js Documentation** — The official docs at **nextjs.org/docs** cover everything about the framework.
- **Tailwind CSS** — Learn how the styling works at **tailwindcss.com/docs**.
- **Framer Motion** — Animation library docs at **framer.com/motion**.
- **Stack Overflow** — Search for your error message; chances are someone has had the same issue.

> **You did it!** By following this guide, you now have a fully functional, deployed learning game that can help children around the world. Take a moment to appreciate what you've built — and then go play it with a child you love. 💜

You're All Set!

You now have everything you need to install, run, and share Wonderful Minds with the world. Every child who plays it will learn, grow, and smile — thanks to you.

Happy playing! 🎮